Learning a distribution

An AI image generator only ever sees a finite set of training images. What should it learn from them so it doesn't just copy them?

Oct 8, 2026 Retrieved Nov 12, 2026

You will learn

- How a network can turn random numbers into new images.
- Why far fewer random numbers than pixels can be enough to make an image.
- Why using so few numbers rules out the most common way of training a model, and what can be done about it.
- What training really produces on a small example, and how to catch a model that copies its training images.
- Where the error of a trained model comes from.

Before you start

- Random variables, densities and expectations
- What a neural network is, and training by gradient descent
- Multivariable calculus for the formal statements, which can be skipped on a first read

Most of the neural networks we meet first are trained to recognize things. We show one an image of a handwritten digit and it tells us which digit it is. Now imagine asking for the opposite. After seeing many handwritten digits, the network should draw new ones that nobody wrote. A network that does this is a *generative model*. The mathematics that makes it work is the subject of this course.

The figure below shows such a model at work. On the left are digits from MNIST, a standard dataset whose training set holds $60{,}000$ handwritten digits. On the right are digits drawn by a generator, our word for a network that draws images, trained on that set. Almost none of them are copies of training digits.

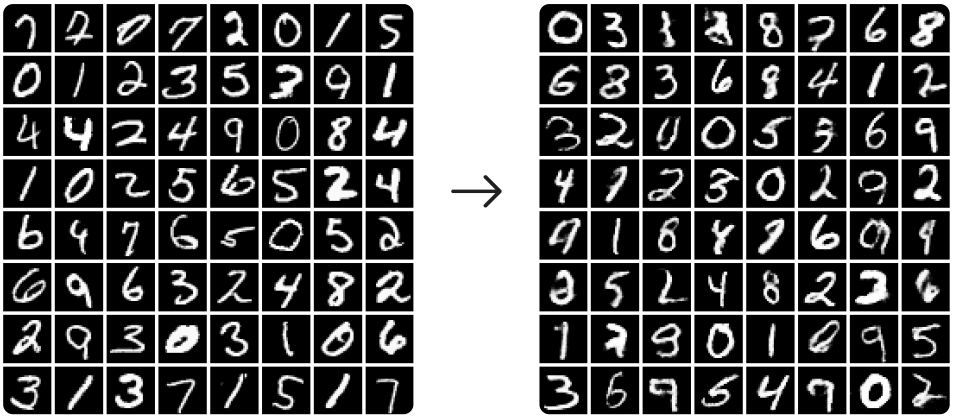


Figure 1. Left, 64 digits from the MNIST training set. Right, 64 digits drawn by a small generator trained on that set (a Wasserstein GAN, the subject of Chapter 5).

That last detail is the problem, because copying would have been easy. We call a model that stores the $60{,}000$ digits and returns one at random the *copying model*. It matches the training set perfectly, yet it is useless, because it can only show digits we already have. What we want is harder to say. A generator should learn the process that produced the digits, with the training set as its only evidence about that process. This chapter is about what that asks of a network.

1 Copying or learning

Before writing anything formal, we want an example we can look at. Images are a poor choice, since each one is a point in a space with hundreds of dimensions where no distribution can be plotted. So this chapter works with a much smaller example in which every sample is visible, namely 12 points drawn at random on the unit circle. Think of the 12 points as our training set and of the circle as the process that produced them, hidden from the model. Figure 2 shows what each model would produce. The copying model returns one of the 12 points, so its samples pile up on them (middle). The model we want produces points anywhere on the circle (right). That is where the 12 came from.

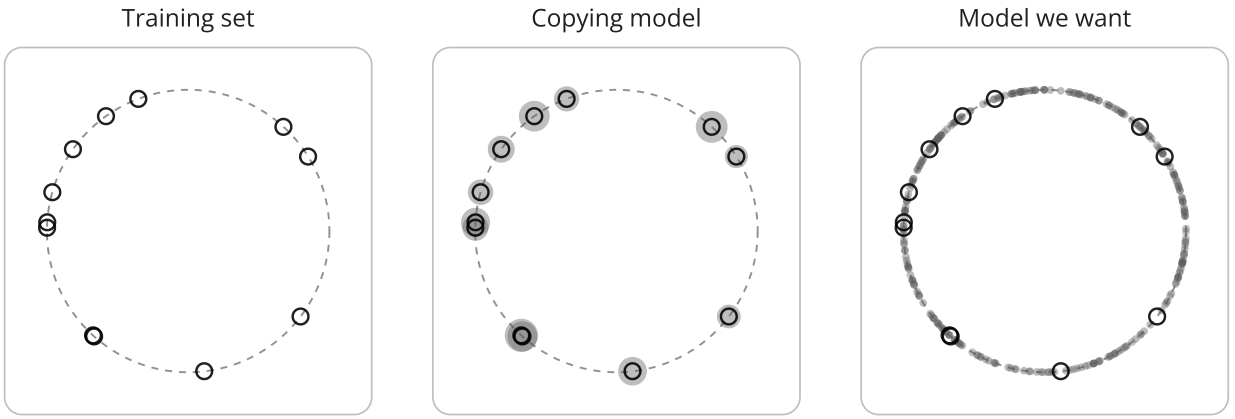


Figure 2. Left, the 12 training points (open circles) on the unit circle (dashed). The model sees the points but never the circle. Middle, 300 samples of the copying model. Each sample is one of the 12 points, so the samples pile up there. The grey disc around a point grows with the number of samples that landed on it. Right, 300 samples of the model we want, spread over the whole circle.

Now the same situation in symbols. We observe n samples $X_1, \dots, X_n$ drawn independently from an unknown probability distribution $\mathbb{P}^*$ on $\mathbb{R}^d$. On the circle, $d = 2$, $n = 12$ and $\mathbb{P}^*$ is the uniform distribution on the unit circle. For images, each sample is a vector with one coordinate per pixel and colour channel, so d counts these coordinates. A grey MNIST digit has $d = 28 \times 28 = 784$ and a CIFAR-10 photo has $d = 32 \times 32 \times 3 = 3072$. All we know about $\mathbb{P}^*$ comes from the samples. The plainest summary of them, which puts equal weight on each one, is defined as follows:

Definition 1 (Empirical distribution)**.** The *empirical distribution* of $X_1, \dots, X_n$ is

$$\mathbb{P}_n = \frac{1}{n} \sum_{i=1}^{n} \delta_{X_i},$$

where δ_x is the point mass at x. Drawing from $\mathbb{P}_n$ means picking one of the training images uniformly at random.

Sampling from $\mathbb{P}_n$ is exactly what the copying model does, so the copying model now has a formula. On the circle, $\mathbb{P}_n$ puts mass $1/12$ on each training point, while $\mathbb{P}^*$ spreads its mass evenly over the whole circle. What we want is a distribution $\widehat{\mathbb{P}}$ built from the samples that is close to $\mathbb{P}^*$ and that we can sample from, since new images are what we are after. To get there we first need a set of candidate networks, since a network has to describe a distribution before we can ask how far it is from $\mathbb{P}^*$.

2 Three ways to describe a distribution

Usually, a neural network computes a function. A distribution on $\mathbb{R}^d$ is another kind of object, which assigns a probability to every set of images. The question is what the network should output so that a distribution follows from it. Three answers are in common use.

The most direct answer asks the network for a *density* $p_\theta(x)$, a function that says how likely each image is. This is harder than it sounds. A density must be non-negative and integrate to 1, but a flexible neural network meets neither condition on its own. Getting a normalized density out of a network constrains it, in how it is built or in how it samples.

Another answer drops the normalization. The network outputs a function that is only proportional to the density, or the gradient of its logarithm in x, which is called the *score*. Neither contains the normalizing constant, so the network has nothing to enforce and the usual image architectures can be used as they are. This freedom has two costs. The model no longer gives probabilities. Drawing a sample also takes many steps that follow the score, because the score only says which way to move. The box below writes out some models behind these two answers, with the score of a Gaussian as an example.

Going further

Writing down a density

An *autoregressive model* orders the d coordinates (one colour channel of one pixel each) and writes the distribution as a product of one-dimensional conditionals,

$$p_\theta(x) = \prod_{i=1}^{d} p_\theta(x_i \mid x_1, \dots, x_{i-1}).$$

Pixel values are discrete, so each factor is a probability table over the 256 intensity levels. Each table sums to 1, so the product does too. Spreading each level uniformly over an interval of width $1/256$ turns it into a density on $\mathbb{R}^d$ (Theis, Oord, and Bethge 2016). The negative log-likelihood is a sum of d classification (cross-entropy) losses, while sampling takes d network calls, one coordinate at a time (Oord, Kalchbrenner, and Kavukcuoglu 2016).

Another model with a normalized density is the *normalizing flow*. It turns a simple random vector into an image through an invertible map, usually a chain of simple invertible layers. Each image can therefore be traced back to the single noise vector that produced it. Formally, the flow uses a *diffeomorphism* $T_\theta : \mathbb{R}^d \to \mathbb{R}^d$, an invertible differentiable map whose inverse is also differentiable. Its noise follows a latent distribution γ with a density $p(z)$. If $Z \sim \gamma$ and $X = T_\theta(Z)$, the change of variables gives

$$\log p_\theta(x) = \log p\big(T_\theta^{-1}(x)\big) + \log \left| \det \frac{\partial T_\theta^{-1}}{\partial x}(x) \right|.$$

The likelihood is computable exactly when the inverse and the determinant are cheap, which constrains the architecture (Rezende and Mohamed 2015; Dinh, Sohl-Dickstein, and Bengio 2017).

An *energy-based model*, by contrast, leaves the density unnormalized. It takes any network $E_\theta : \mathbb{R}^d \to \mathbb{R}$ and sets $p_\theta(x) \propto e^{-E_\theta(x)}$, so the architecture is free. The cost moves to the normalizing constant $\int e^{-E_\theta(x)}\, dx$, an integral over all of $\mathbb{R}^d$ that has no closed form for a general network. In high dimension it can only be estimated by sampling from the model.

The *score* avoids this constant. To see how, take the standard Gaussian in one dimension. There $\log p(x) = -x^2/2 - \tfrac{1}{2}\log(2\pi)$, so the score is $-x$. At $x = 2$ it equals -2 and points back toward the centre, with a pull that grows the further we go. In general the score points toward images of higher density. The normalizing constant (here the term with $\log(2\pi)$) vanishes in the gradient. For an energy-based model the score is $\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$, a vector of the same size as the input that a network can output directly. A network can therefore be trained to estimate the score of the data from samples (Hyvärinen 2005). The diffusion models of Chapter 7 learn the score of the data blurred by noise at many levels (Vincent 2011; Song and Ermon 2019).

The last answer goes straight to what we are after, which is new samples. It leaves the density aside and describes how a sample is made. A sample starts as random noise from a simple distribution, and a network then turns this noise into an image in a single step. This is the answer the rest of the chapter develops. The score answer ends the same way, since following the score also turns noise into an image, only in many small steps.

3 Generators and the pushforward

Each of these two steps gets a name. The noise is a random vector Z with a fixed distribution γ on $\mathbb{R}^k$, usually the standard Gaussian $\mathcal{N}(0, I_k)$, which is easy to sample and has no parameters. We call Z the *latent variable*, since it never appears in the data. The network that turns Z into an image is the *generator* $g : \mathbb{R}^k \to \mathbb{R}^d$, a neural network with weights θ. A definition like this stays abstract until we see a generator written out. The circle lets us write one that produces $\mathbb{P}^*$ exactly.

Example 2 (A generator for the circle)**.** Take $k = 1$, let $Z \sim \mathcal{N}(0, 1)$, and define g by

$$g(z) = \big(\cos 2\pi\Phi(z),\, \sin 2\pi\Phi(z)\big),$$

where Φ is the distribution function of the standard Gaussian. The random variable $\Phi(Z)$ is uniform on $[0, 1]$, since $P(\Phi(Z) \le u) = P\big(Z \le \Phi^{-1}(u)\big) = \Phi\big(\Phi^{-1}(u)\big) = u$ for every u in $(0, 1)$, so the angle $2\pi\Phi(Z)$ is uniform on $[0, 2\pi)$ and $g(Z)$ is uniform on the unit circle. Each draw of Z gives a new point of the circle. Taken over all draws, the distribution of $g(Z)$ is exactly $\mathbb{P}^*$.

In the example, the generator turned a Gaussian on a line into a uniform distribution on a circle. We found that distribution without writing a density for it. The distribution of $g(Z)$ is what training will adjust. Like any distribution, it is known once we know the probability of each set A of images. Here that probability comes from the noise. The image $g(Z)$ lands in A exactly when Z lands among the latent points that g sends into A, so the two events have the same probability.

Definition 3 (Pushforward)**.** Let $Z \sim \gamma$ on $\mathbb{R}^k$ and let $g : \mathbb{R}^k \to \mathbb{R}^d$ be measurable. The *pushforward* of γ by g is the distribution of $g(Z)$, written $g_{\#}\gamma$:

$$(g_{\#}\gamma)(A) = \gamma\big(g^{-1}[A]\big) \quad \text{for every measurable set } A \subset \mathbb{R}^d.$$

We write $\mathbb{P}_g = g_{\#}\gamma$.

Here $g^{-1}[A] = \{z \in \mathbb{R}^k : g(z) \in A\}$ is that set of latent points, called the *preimage* of A. The square brackets mark that g^{-1} is applied to a set. The preimage exists for every g, invertible or not, since it only collects the latent points that g sends into A. *Measurable* is a technical condition satisfied by all continuous functions and all sets in this series. For the circle generator,

$\mathbb{P}_g = \mathbb{P}^*$. Figure 3 applies the definition to an arc A of the circle. The samples that land in A are the dark ticks on the right. Their latent values fill one interval of the line on the left, the preimage $g^{-1}[A]$ (lower arrow). The shaded area above that interval is the probability of the arc.

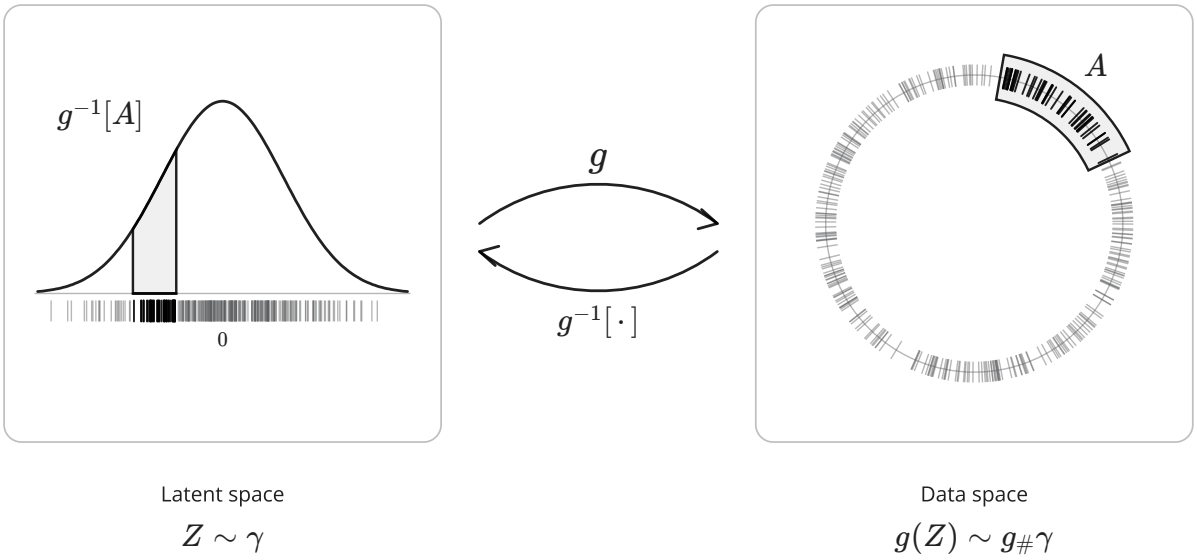


Figure 3. The circle generator of Example 2 pushes the Gaussian γ on the line forward to the uniform distribution on the circle. Each tick is a sample $Z \sim \gamma$ on the left and its output $g(Z)$ on the right. The samples that land in the region A (dark ticks) are exactly those whose Z falls in the preimage $g^{-1}[A]$, which here is an interval. Both carry the same probability $(g_{\#}\gamma)(A) = \gamma(g^{-1}[A])$, the shaded area under the density. The lower arrow goes back from a set to its preimage.

In practice the pushforward is easy to use. To sample, we draw $Z \sim \gamma$ and compute $g(Z)$. Each new draw of Z gives a new image while g stays fixed. Changing the weights θ changes g, and with it the distribution of $g(Z)$. The distribution of Z is simple and never changes, so everything training learns sits in the weights of g, an ordinary deterministic function. Learning a distribution therefore comes down to adjusting the weights of a function, the kind of problem gradient descent is built for. The generators obtained for all values of θ form the set of candidates we asked for. We call this set the *model class* $\mathcal{G}$. Learning means choosing $\widehat{g} \in \mathcal{G}$ so that $\widehat{\mathbb{P}} = \mathbb{P}_{\widehat{g}}$ is close to $\mathbb{P}^*$.

One choice is still open, the latent dimension k. The circle generator needs a single latent number to produce a curve in the plane. One might expect a 784-pixel digit to need 784 random numbers. Far fewer are enough, because images are far from arbitrary points of $\mathbb{R}^d$.

4 Images lie near a manifold

To see why, compare an MNIST digit with an image of the same size drawn at random. An image whose $28 \times 28 = 784$ pixels are drawn independently and uniformly at random looks like noise. Natural images obey regularities that such random images almost never satisfy. An empty background around a centred digit, similar values in neighbouring pixels and ink that forms a single connected stroke of roughly constant width are a few examples. Each of them is a strong constraint. Together they confine natural images to a tiny part of $\mathbb{R}^d$.

Within that part, natural images can still change, but only along few directions. For instance, if we change the slant of a handwritten 4 or the thickness of its strokes within limits, every intermediate image is still a plausible 4. Most other directions of $\mathbb{R}^d$, such as adding independent noise to every pixel, quickly leave the set of plausible digits. We have met this shape before. The circle sits in the plane, yet a single angle locates a point on it. Natural images lie near a surface of the same kind, thin and of low dimension inside $\mathbb{R}^d$, which mathematicians call a manifold. Figure 4 draws this picture for digits. Along the path on the sheet, a 4 changes its shape step by step and stays plausible. The images off the sheet have left the manifold. Two of them still come from the generator itself, which follows $\mathcal{M}$ near the training digits and leaves it farther away.

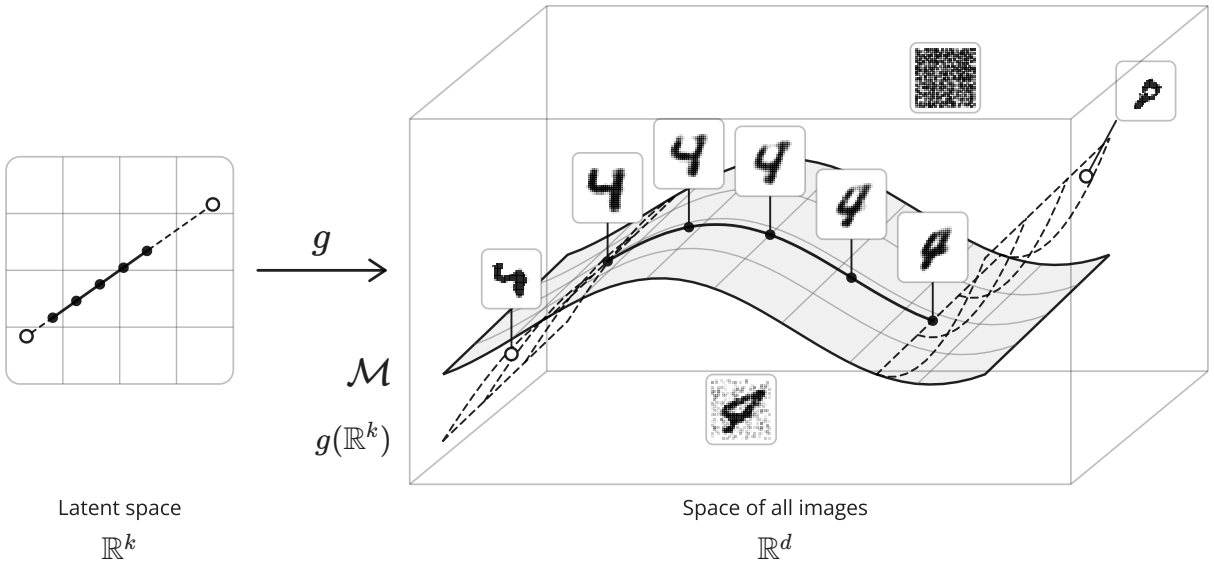


Figure 4. The data manifold $\mathcal{M}$ (shaded) and the set $g(\mathbb{R}^k)$ of outputs of a generator g, dashed where it leaves $\mathcal{M}$. A straight segment in $\mathbb{R}^k$ (left) maps to a path of plausible 4s where the two agree. Latent points further along the same line (open circles) land where $g(\mathbb{R}^k)$ leaves $\mathcal{M}$, giving an implausibly heavy 4 and a swollen shape. A noisy 4 and random pixels lie off both.

To use the idea of a manifold, we need to say precisely what one is. The requirement is local. Near any point of the surface, a few numbers should be enough to say where we are, and changing those numbers should move us smoothly along it.

Definition 4 (Manifold). A set $\mathcal{M} \subset \mathbb{R}^d$ is an *m-dimensional manifold* if every point of $\mathcal{M}$ has a neighbourhood in $\mathcal{M}$ (the part of $\mathcal{M}$ inside a small ball around the point) described by m coordinates. This means there is a continuously differentiable invertible map φ from an open subset of $\mathbb{R}^m$ onto the neighbourhood, with a continuous inverse and a derivative of rank m at every point.

In short, our claim about images reads as follows.

> **Assumption 5** (Manifold hypothesis)**.** The distribution $\mathbb{P}^*$ of natural images concentrates near a manifold $\mathcal{M}$ whose dimension m is much smaller than d.

By *concentrates near* we mean that most of the probability lies within a small distance of $\mathcal{M}$. The dimension m of $\mathcal{M}$ is called the *intrinsic dimension* of the data. Estimating it from a real dataset checks the hypothesis. Estimators rely on the fact that in dimension m the number of data points within distance r of a point grows like r^m as r increases. This holds for r small enough that $\mathcal{M}$ looks flat at that scale, but larger than the spread of the data around $\mathcal{M}$. Pope et al. (2021) report estimates of 7 to 13 for MNIST ($d = 784$), 13 to 26 for CIFAR-10 ($d = 3072$) and 26 to 43 for ImageNet ($d = 224 \times 224 \times 3 = 150{,}528$).[1] In every case m is a few dozen at most, whereas d runs from hundreds to hundreds of thousands.

This is what makes a small latent dimension possible. With $k \geq m$ latent dimensions, $g(z)$ can move along $\mathcal{M}$ as z moves in $\mathbb{R}^k$, as the circle generator does with $k = m = 1$. A generator with $k < d$ then puts all its mass on a set of dimension at most k, a thin subset of $\mathbb{R}^d$. The next section shows what that costs.

5 Why maximum likelihood fails for generators

Putting all the mass on a thin surface does not stop a generator from representing $\mathbb{P}^*$. The circle generator represents it exactly. We cheated a little, though. We wrote it down from $\mathbb{P}^*$, which is never known in practice. The real difficulty is to learn a generator from the samples alone. The most common way to do so, maximum likelihood, needs a density. A distribution $\mathbb{P}$ on $\mathbb{R}^d$ has a *density* p when the probability of every region A is the volume integral $\mathbb{P}(A) = \int_A p(x)\,dx$, where p is a function with values in $[0, +\infty)$. Maximum likelihood raises $p_\theta(X_i)$ at every training image X_i, so without a density it has nothing to raise.

Whether $\mathbb{P}_g$ has a density depends on the set $g(\mathbb{R}^k)$ of all outputs of the generator. When $k < d$ it has none. The argument takes four steps.

1. The usual networks stretch distances by a bounded factor. A map g is *Lipschitz* with constant L if $\|g(z) - g(z')\| \leq L\|z - z'\|$ for all z, z', so two points never end up more than L times farther apart than they started. Networks built from linear layers and the usual activations (ReLU, SiLU, tanh) are Lipschitz.
2. A Lipschitz generator with fewer latent dimensions than pixels, $k < d$, has a set of outputs $g(\mathbb{R}^k)$ of volume zero in $\mathbb{R}^d$. For the circle generator seen before, $g(\mathbb{R})$ is a curve, and a curve has no area.
3. $\mathbb{P}_g$ puts all its probability on $g(\mathbb{R}^k)$, since every output of g lies there, whatever the distribution γ of Z.
4. Suppose $\mathbb{P}_g$ had a density p. The probability of $g(\mathbb{R}^k)$ would then be the integral of p over $g(\mathbb{R}^k)$. By step 2 this set has volume zero, so the integral is 0, whatever p is. By step 3 the same probability is 1. Both cannot hold, so $\mathbb{P}_g$ has no density.

Step 2 is the one that needs work. It holds because $g(\mathbb{R}^k)$ can be covered by small boxes whose total volume is as small as we like. The box below works this out.

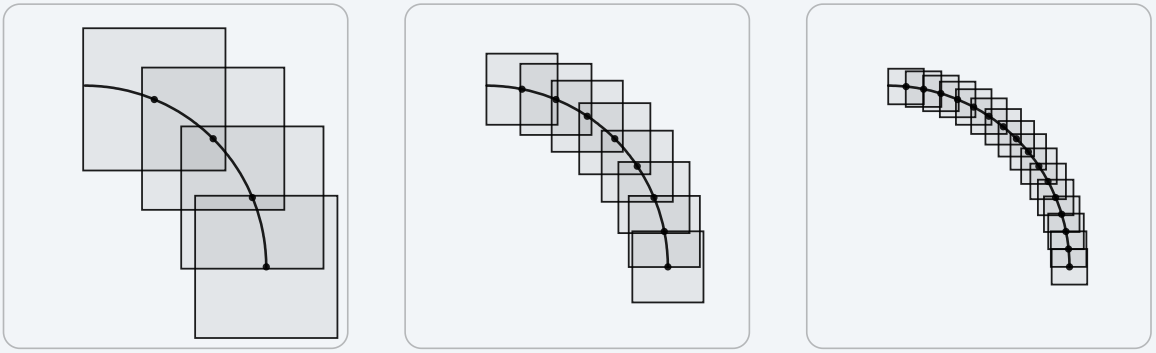
Going further

Why step 2 holds

Start with the simplest case $k = 1$ and $d = 2$, a generator that maps a segment of length 1 to a curve in the plane. The plan is to cover the curve with small squares and show that their total area can be made as small as we like. If we split the segment into N pieces of length $1/N$, each piece becomes a stretch of curve whose points all lie within L/N of its starting point, because the network stretches distances by at most L. That stretch of curve fits inside a square of side $2L/N$ centred on the starting point. Together the N squares cover the whole curve, with total area

$$N \times \left(\frac{2L}{N}\right)^2 = \frac{4L^2}{N}.$$

A finer split means more squares, but their number grows like N while the area of each one falls like $1/N^2$. The total shrinks. Doubling N halves it, so squares of total area as small as we like still cover the curve. The curve has no area.



$N = 4$ total area 2.47 | $N = 8$ total area 1.23 | $N = 16$ total area 0.62

Figure 5. The curve is a quarter of the unit circle. It is the image of $[0, 1]$ under $g(t) = (\cos \frac{\pi t}{2}, \sin \frac{\pi t}{2})$, with $L = \pi/2$. Each panel covers it with N squares of side $2L/N$, centred on the starting points of the pieces (dots). Each doubling of N halves the total area π^2/N of the squares while the curve stays covered. The curve therefore has area zero.

The same count works in any dimension. Take a unit cube in $\mathbb{R}^k$ and split it into N^k small cubes of side $1/N$. Each small cube is sent inside a box of volume proportional to N^{-d}, since each of the d sides of the box shrinks like $1/N$. Together these boxes cover the image of the cube with a total volume proportional to N^{k-d}, which goes to zero whenever $k < d$. With $k = 1$ and $d = 2$ the exponent is -1, the halving we just saw. The number of pieces grows with the latent dimension and the volume of each box shrinks with the data dimension, so $k < d$ is what makes the total vanish. For $k \geq d$ the count says nothing. A set that can be covered by an arbitrarily small total volume has *volume zero* (Lebesgue measure zero), like a curve in the plane. The image of one unit cube is such a set.

The latent space is unbounded. One cube is not yet all of $g(\mathbb{R}^k)$, and two more facts close the gap. The space $\mathbb{R}^k$ is the union of countably many unit cubes, one for each point with integer coordinates. Its image $g(\mathbb{R}^k)$ is the union of their countably many images, each of volume zero. A countable union of sets of volume zero has volume zero too. To see why, fix a target volume, however small. Cover the first image with boxes of total volume at most half the target. Each later image gets boxes of at most half the total volume allowed to the image before it. All these boxes together cover $g(\mathbb{R}^k)$ with a total volume below the target.

The count on each cube used the stretching bound on that cube only. A cube far from the origin may need a larger constant than L, which makes its boxes larger for a given N but still lets their total volume go to zero. This is why a generator that is only *locally Lipschitz*, with a bound on every bounded region but no single bound for all of $\mathbb{R}^k$, still has a set of outputs of volume zero.

Step 2 needs both of its hypotheses. Continuity alone would not be enough. The space-filling curves of Peano and Hilbert are continuous, yet they pass through every point of a whole square. Nor can $k < d$ be dropped. When $k = d$ and g is a diffeomorphism, a change of variables turns the integral $\gamma(g^{-1}[A])$ over $\mathbb{R}^k$ into an integral over A. A normalizing flow is a generator of exactly this kind, with $k = d$ and a diffeomorphism as its map. This change of variables is how it gets its density. With $k < d$ no such change of variables exists. The Lipschitz hypothesis can still be weakened a little. The covering argument only needs the Lipschitz bound on each bounded region, a weaker condition called *locally Lipschitz* that also covers networks whose stretching grows with the input. We can now state the result in full.

> **Proposition 6** (No density with fewer latent dimensions than pixels). *Let $g : \mathbb{R}^k \to \mathbb{R}^d$ be locally Lipschitz with $k < d$. Then the set $g(\mathbb{R}^k)$ has volume zero in $\mathbb{R}^d$. Consequently, for every distribution γ on $\mathbb{R}^k$ the pushforward $\mathbb{P}_g = g_{\#}\gamma$ has no density on $\mathbb{R}^d$.*

Arjovsky and Bottou (2017) prove a version of this result for networks built from affine layers and the usual activations. They use it to explain why GAN training is unstable.

A distribution without a density may look broken, but we have met two already. The empirical distribution $\mathbb{P}_n$ puts all its probability on n points, a set of volume zero, so it has no density on $\mathbb{R}^d$. Each of its point masses is the Dirac delta of physics, which no density can describe either. The other is the uniform distribution on the circle, our $\mathbb{P}^*$. It has a density along the circle, $1/(2\pi)$ per unit of length, but none in the plane. Whether a distribution has a density therefore depends on the space in which volume is measured. A generator with $k < d$ has a probability distribution like any other, which only lacks a density on $\mathbb{R}^d$. Maximum likelihood needs exactly that density.

Training such a generator therefore needs either a way to give it a density, for instance by adding noise to its output as Chapter 3 does, or a way to compare distributions without one.

6 What training produces

The sections so far say what a generator can represent. They say nothing about what training will produce. To find out, we train generators on a few points of the circle and look at every sample. The question is whether they learn the circle or the points.

We start with the case of the previous section. Each generator maps one latent number to a point of the plane, as the circle generator does. Since $k = 1$ is smaller than $d = 2$, Proposition 6 says that none of them has a density. Each is a network with two hidden layers of the same width and SiLU activations. We compare a narrow generator of width 16 with a wide one of width $1{,}024$.

Training needs a number to minimize. Here it is the maximum mean discrepancy (MMD) between the samples and the training points. MMD compares two clouds of points through a similarity kernel, a function κ of two points that is largest when they coincide and falls to nearly zero when they are far apart. It averages this similarity over pairs within each cloud and over pairs across the two clouds. The MMD is small when the two kinds of average agree.

The smallest case shows how this behaves. Put one point x in the first cloud and one point y in the second. Then $\mathrm{MMD}^2 = \kappa(x, x) + \kappa(y, y) - 2\kappa(x, y)$. The two terms within the clouds are fixed at the largest value of the similarity. When $x = y$ the term across the clouds cancels them and the result is 0. As y moves away that term fades, so the result rises toward twice the largest value. Larger clouds use the same three averages. Chapter 2 gives their formula. Our kernel is a sum of five Gaussian kernels whose widths range from 0.02 to 0.5, so that both near and far samples feel a pull. With such kernels, MMD amounts to blurring each cloud into a smooth density and measuring how much the two blurred densities differ. The losses below report its square, MMD^2.

I chose the MMD because it needs neither a density nor a second network trained alongside the generator, which some models of later chapters use. Our generators have no density. This is the kind of comparison the previous section asked for. Without a second network, the only training that can go wrong is the generator's own.

Every generator trains for $100{,}000$ steps. Every setting is repeated three times, each time with its own training points drawn from the circle, so a result that holds in all three does not depend on one lucky draw.

A low training objective alone cannot tell us much, because the copying model is exactly what scores best on it. We judge each generator on $2{,}000$ of its samples with three numbers.

- The *training loss* is the objective itself, the MMD between the samples and the training points.
- The *test loss* is the same MMD, measured against $2{,}000$ fresh points from the circle that the generator never saw. It estimates how far the generator is from $\mathbb{P}^*$. Fresh points of the circle score 0.002 themselves, because $2{,}000$ points are only a sample.

- *Copies* is the share of samples that land very close to a training point. Picture a small disc around each training point, with a radius chosen so that a fresh point of the circle falls in one of the discs only 1% of the time. A sample inside a disc counts as a copy, so fresh points score about 1% by construction. The idea of judging a sample by its distance to the training set, against the same distance for fresh points, comes from the data-copying test of Meehan, Chaudhuri, and Dasgupta (2020).

Before looking at the table, it helps to say what success would look like. A generator that has learned the circle behaves like fresh points of the circle, so its test loss should be close to 0.002 and its copies close to 1%. Its training loss should be about the same as that of the true circle, the column *Truth* below. A generator that has learned the points should beat the truth on training loss, with a test loss near *Truth* and many more than 1% copies. The copying model shows why its test loss sits near *Truth*. Its samples are the training points themselves, so its test loss compares the training points with fresh points of the circle. *Truth* makes the same comparison, which is why a generator close to copying scores about *Truth* on the test loss.

6.1 Twelve points

Width	Points	Training loss	Truth	Test loss	Copies
16	12	0.097–0.205	0.351–0.600	0.292–0.490	1.6–3.2%
1024	12	0.017–0.020	0.351–0.600	0.264–0.537	6.7–24.1%

Circle generators of widths 16 and 1024 with one latent dimension, trained on 12 points for 100,000 steps. Each range gives the smallest and largest value over the three draws of the training points. *Truth* is the training loss of 2,000 points of the true circle against the same training points.

Start with the training loss. Every generator trained on 12 points scores better on its training objective than the true circle does. A perfect generator draws its samples from $\mathbb{P}^*$ itself, yet on the same points its training loss is at least 0.35. Every trained generator, of either width, scores at most 0.21. Twelve points cover little of the circle, so a model that hugs them beats the truth on a score that only sees them.

Before running the experiment, I expected a wider generator to trace the circle more closely, since it can bend more freely. It traced the training points more closely. In the draw shown in Figure 6, the generator of width $1{,}024$ runs along straight chords from one training point to the next and puts most of its samples near the points. The narrow one follows the circle more closely between the points. Both curves leave part of the circle uncovered. In each of the three draws the wider generator copies more. Neither is close to the circle. In every draw the test loss of both widths stays above 0.25, far from the 0.002 of fresh points.

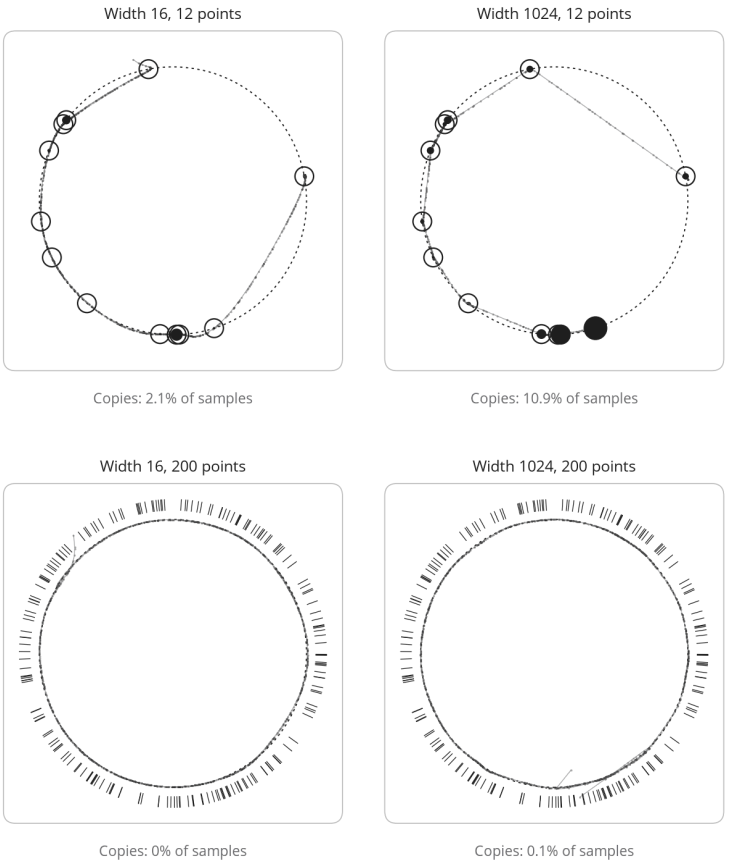


Figure 6. Generators with one latent dimension, second draw of the training points. The buttons switch between 12 training points (open circles) and 200 (ticks outside the circle). Grey dots are 1,200 of the 2,000 samples. A faint line joins them in the order of their latent values and traces the curve $g(\mathbb{R})$. The dashed line is the true circle. Copies sit too close to their training point to be seen, so the dark disc on each point grows with their number. The share of copies is given under each panel. *The online version of this figure is interactive.*

Some samples between the training points are forced on any continuous generator. Its outputs form one connected curve. That curve has to travel from one training point to the next. The latent values along the way give samples between the points. The box below makes this precise.

Going further

Why a generator cannot copy exactly

A set is *connected* if it cannot be split into two pieces that are separated from each other. The space $\mathbb{R}^k$ is connected, and so is $g(\mathbb{R}^k)$ for a continuous g. A Gaussian gives positive probability to every open set, so $\mathbb{P}_g$ puts mass near every point of the connected set $g(\mathbb{R}^k)$. With $n \geq 2$ distinct training points, $\mathbb{P}_n$ sits on n separated points, which a continuous generator fed with Gaussian noise cannot produce exactly. It can still come arbitrarily close in the Wasserstein distance of Chapter 2, by sending almost all of its mass near the training points and keeping thin threads in between.

Now compare this with what we expected. On 12 points both widths copy more than the 1% of fresh points, the wide one up to 24%. Their test loss is of the same order as *Truth* and more than a hundred times the 0.002 of fresh points. Together with the training loss, all three

numbers say the same thing. These generators learned the points and the chords between them. They did not learn the circle.

By training loss, the wide generator looks better, since it reaches the lower training loss in each draw. It is also the generator that comes closer to copying. The reason lies in the objective, which compares the generator with $\mathbb{P}_n$. The copying model minimizes it. Every step toward the copying model lowers it. Twelve points do not show that the data lie on a circle, so the objective built from them rates the chords above the circle, as the training losses show. This failure is called *memorization*.

6.2 Longer training

Next we follow the same runs through training, to see what the extra steps did. I expected the extra steps to help, or at worst to change little once the training loss flattened. It never flattened. At $90{,}000$ steps it was still falling in every draw.

With one latent number, longer training moves the generator slightly further toward copying. The figure below follows the wider generator through training, one panel per draw of the 12 points. Between $6{,}000$ and $100{,}000$ steps its training loss keeps falling as it moves toward the copying model, while its test loss grows by 0.02 to 0.04. The rise is small, about the size of the wobble in the second draw, but the generator after $6{,}000$ steps has the lower test loss in all three draws. We use $6{,}000$ steps as one early checkpoint. It is not the best one, since the lowest test loss comes at $16{,}000$, $8{,}000$ and $26{,}000$ steps in the three draws.

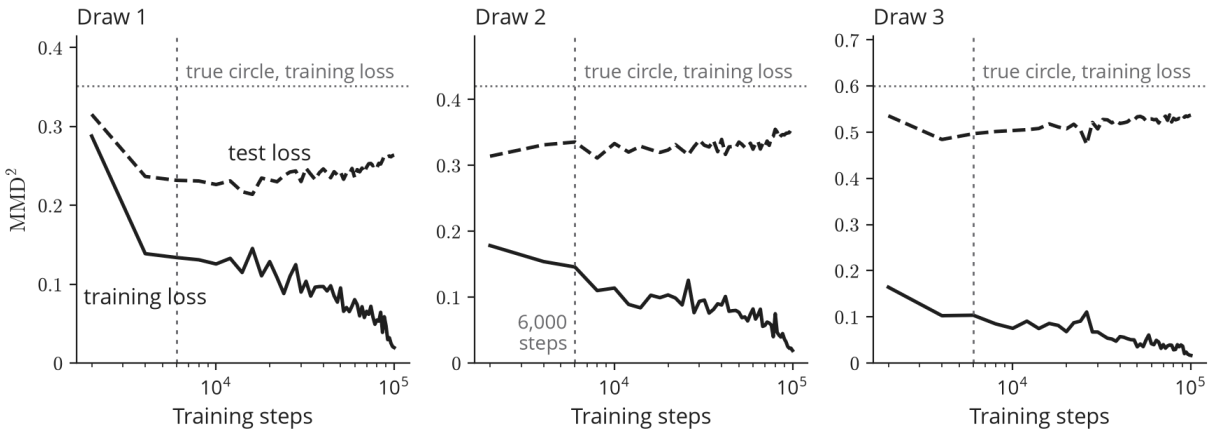


Figure 7. Training loss (solid) and test loss (dashed) of the width-1024 generator with one latent dimension during training, one panel per draw of the 12 training points. The dotted line is the training loss of the true circle on the same points. The vertical line marks 6,000 steps. Between $6{,}000$ and $100{,}000$ steps the training loss falls from between 0.10 and 0.15 to about 0.02. Between $70{,}000$ and $90{,}000$ steps it still falls by 28%, 36% and 9% in the three draws.

6.3 More points

More data cures the copying. With more points the blurred training cloud comes close to the blurred circle, so the objective has less reason to favour the points. On 200 points both widths copy at most 0.1% of their samples. Their samples follow the circle, as the second tab of Figure 6 shows.

Width	Points	Training loss	Truth	Test loss	Copies
16	200	0.012–0.015	0.016–0.025	0.008–0.017	0–0.1%
1024	200	0.005–0.006	0.016–0.025	0.012–0.026	0–0.1%

The same generators on 200 points.

The test loss falls from 12 to 200 points in every draw, but it stays close to *Truth*, which falls too. A copying model would score about *Truth* as well, so the test loss alone cannot tell a generator that follows the circle from one that copies. The copies can tell them apart, and so can the distance from each sample to the circle, which we measure next.

The cure is only partial. On 200 points both widths still beat *Truth* on training loss, while their test loss is still 4 to 13 times the 0.002 of fresh points. Two hundred points leave a visible gap. Their copy rate is also below the 1% of fresh points, a sign that the samples sit slightly off the circle. Their median distance to it is 0.0015 to 0.0043. That looks small, but in each draw it is at least nine times the radius of the discs that define a copy, which is 0.00013 to 0.00018 on 200 points.

6.4 Two latent dimensions

A curve is a thin object, so one may wonder whether memorization comes from this restriction. We repeated the runs with $k = d = 2$, where Proposition 6 no longer applies and a generator can spread its mass over an area of the plane. Each result of the runs with one latent dimension holds again in all three draws. Longer training costs more here. Between $6{,}000$ and $100{,}000$ steps the test loss of the wide generator on 12 points grows by 0.05 to 0.08. On 200 points both widths copy at most 0.1% of their samples, with a median distance to the circle of 0.0034 to 0.0058. The Lab box below lists every run.

The samples are no longer tied to a curve. On 12 points the wide generator still puts most of them on the training points and on the straight chords between neighbouring points. The narrow one follows the same chords more loosely and scatters more samples inside the circle.

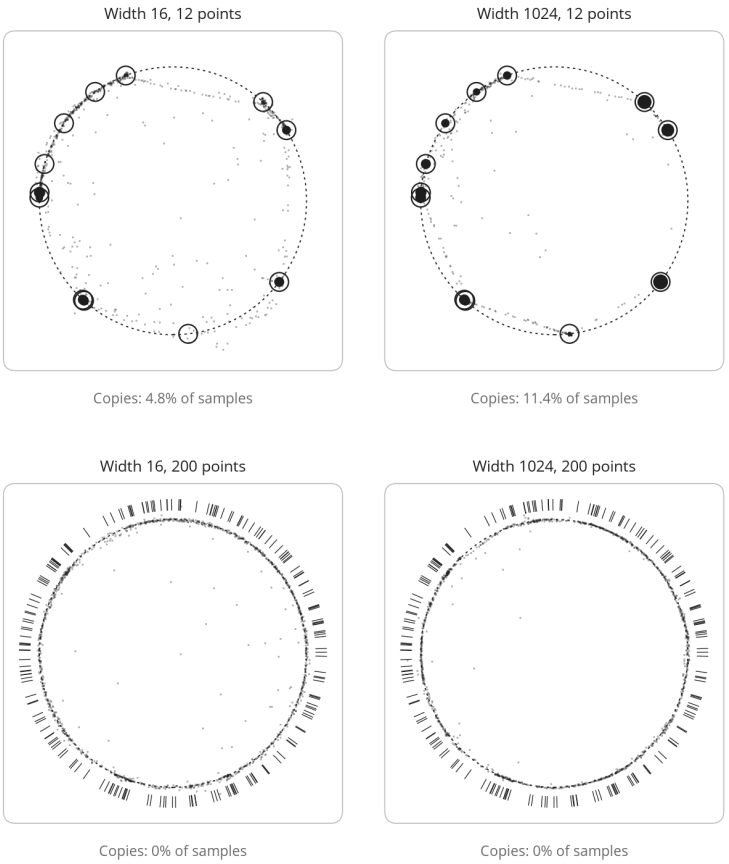


Figure 8. Generators with two latent dimensions, first draw of the training points. The buttons switch between 12 training points and 200. Markers as in Figure 6, without the curve, which exists only for one latent dimension. *The online version of this figure is interactive.*

With two latent dimensions we also trained the wider generator on 25, 50, 100 and 200 points to see how the cure sets in. With 25 points its share of copies is already close to the 1% of fresh points (1.0% to 2.5%) and falls further with more points.

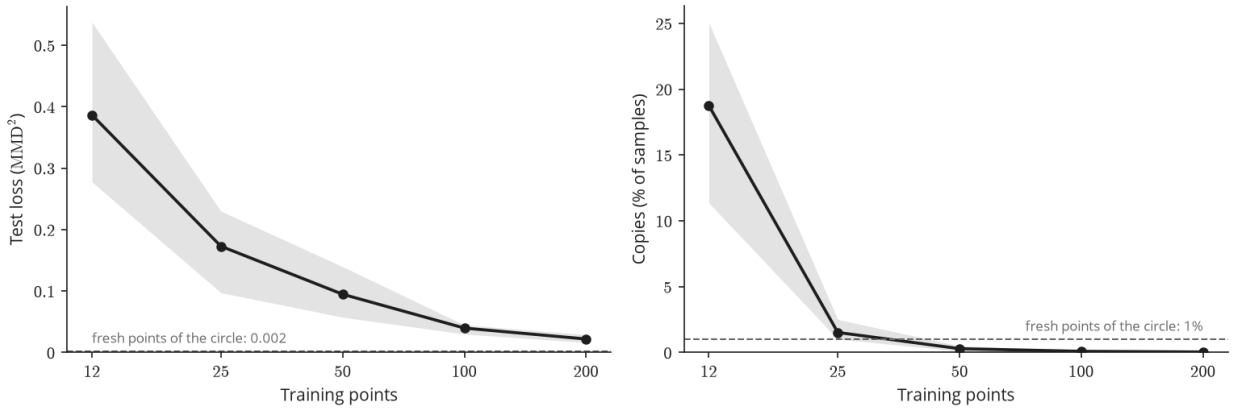


Figure 9. Test loss (left) and copies (right) of the width-1024 generator with two latent dimensions against the number of training points. Dots are the mean of the three draws, and the band spans the smallest and largest value. Dashed lines mark what fresh points of the circle score.

6.5 Back to the digits

All three numbers need only a held-out set, so later chapters report them on real images too. For the MNIST generator of the opening figure the share of copies is 0.20%, which is why we could say that almost none of its digits are copies. Real test images that were not used to size the discs score 1.15%. The generator sits below them. That is a warning sign of its own. Its digits sit slightly farther from the training images than real digits do, which means they lie slightly off the data.

Copies also appear in large models trained on photographs. Carlini et al. (2023) recover individual training images from diffusion models, and Somepalli et al. (2023) find generated images that reproduce parts of training images.

Lab

Full results

Each setting was trained for $100{,}000$ steps with Adam, at a learning rate of 10^{-3} held for $90{,}000$ steps and then decreased to zero, and repeated with three draws of the training points. A range gives the smallest and largest of the three values. *Truth* is the training loss of $2{,}000$ points of the true circle against the same training points. The runs with one latent dimension are in the first two tables of this section. The table below lists every run with two latent dimensions.

Width	Points	Training loss	Truth	Test loss	Copies
4	12	0.116–0.214	0.351–0.600	0.183–0.472	1.0–4.0%
16	12	0.054–0.106	0.351–0.600	0.256–0.518	3.6–6.6%
64	12	0.016–0.062	0.351–0.600	0.261–0.536	4.5–15.7%
256	12	0.014–0.020	0.351–0.600	0.288–0.507	9.4–23.5%
1024	12	0.010–0.015	0.351–0.600	0.277–0.538	11.4–25.1%
1024	25	0.014–0.018	0.131–0.241	0.097–0.230	1.0–2.5%
1024	50	0.010–0.012	0.063–0.152	0.057–0.139	0.1–0.5%
1024	100	0.007–0.008	0.035–0.053	0.030–0.045	0.1%
1024	200	0.004–0.007	0.016–0.025	0.016–0.029	0–0.1%
16	200	0.005–0.007	0.016–0.025	0.011–0.032	0%
4	200	0.013–0.018	0.016–0.025	0.011–0.030	0%

With two latent dimensions, width 4 has the lowest test loss of the five widths in every draw. Widths 4, 16 and 64 copy the least, in that order. The order also holds with a kernel twice narrower or twice wider (one draw).

In the first draw the width-16 generator with $k = 1$ on 12 points put 31% of its samples farther than 0.25 from every fresh point of the circle. The same run has the highest training loss of the $k = 1$ runs, 0.205. In the other $k = 1$ runs this share is at most 2.4%.

7 Four sources of error

The circle runs missed $\mathbb{P}^*$ in different ways. The wide generator copied 12 points and longer training made it worse, while more points fixed it. To compare such failures, and later those of other models, we need a common statement of the problem and a split of the error into its sources. The statement comes first.

Training on the circle required three choices. One is the model class $\mathcal{G}$ from the section on generators, the set of candidates that training picks from. To pick well we need a way of saying how far the distribution of a candidate's images is from the distribution of real images. That distance usually cannot be computed, so we also need a number computed from the training images that stands in for it. The statement below collects them with their symbols.

Problem 7 (Learning a generative model)**.** We are given n samples $X_1, \dots, X_n$ of an unknown $\mathbb{P}^*$, a model class $\mathcal{G}$ of generators and a way $D(\mathbb{P}^* \,\|\, \mathbb{P}_g)$ to measure how far the model is from the data (a distance or a divergence, which Chapter 2 compares). We look for $\hat{g} \in \mathcal{G}$ such that $D(\mathbb{P}^* \,\|\, \mathbb{P}_{\hat{g}})$ is small. Since $\mathbb{P}^*$ is unknown, $\hat{g}$ is obtained by minimizing an objective $S_n(g)$ computed from the samples.

The problem asks for a small error. Making it small starts with knowing where the error of a trained model comes from. Two quantities describe that. Write $R(g) = D(\mathbb{P}^* \,\|\, \mathbb{P}_g)$ for the error of a generator g. On the circle D is the squared MMD, so the test loss is our estimate of R. The objective S_n depends on the n samples. We write $S(g)$ for the value it would take with infinitely many samples. On the circle, S_n compares the samples of the generator with the 12 training points and S compares them with the whole circle.

With R and S we can follow the path from the data to the trained model. Four generators of the class lie on it, the trained model being the last. Each one marks something we had to give up. The best one is $g_{\mathcal{G}}$, which minimizes R over $\mathcal{G}$. It is what we would pick if we could compute R. Since we usually cannot, we settle for the generator g_S that minimizes S. Since we only have n samples, we settle again for g_{S_n}, which minimizes S_n. And since training may stop before reaching it, what it returns is $\hat{g}$.

Generator	How it is obtained	What we gave up to get it
$g_{\mathcal{G}}$	minimizes R over $\mathcal{G}$	every distribution outside $\mathcal{G}$, and R usually cannot be computed
g_S	minimizes S	the exact distance D
g_{S_n}	minimizes S_n	infinitely many samples
$\hat{g}$	returned by training	an exact minimization

The error of the trained generator splits into four terms,

$$R(\hat{g}) = R(g_{\mathcal{G}}) + \big[R(g_S) - R(g_{\mathcal{G}})\big] + \big[R(g_{S_n}) - R(g_S)\big] + \big[R(\hat{g}) - R(g_{S_n})\big].$$

The sum telescopes and every term is measured with the same D, so the four terms add up to the total error. Each row of the table after the first contributes one term, while the first contributes the error that remains even with the best generator of the class. Of the three terms from the later rows, only the surrogate term is sure to be non-negative, so giving something up does not always raise the error.

The split is an identity, so it holds for any four generators. Their definitions matter only for the signs of the terms. Even there a difficulty hides, since a minimum need not be attained. The box “Why a generator cannot copy exactly” shows that a continuous generator fed with Gaussian noise can come arbitrarily close to the copying model in the Wasserstein distance without producing it. With Gaussian kernels a small Wasserstein distance forces a small MMD. So if the class can approach the copying model in this way, no generator of the class minimizes S_n. In that case g_{S_n} stands for a generator whose objective lies within a small tolerance of its infimum, the value the objective approaches without reaching it, and the same convention applies to $g_{\mathcal{G}}$ and g_S. Every statement about signs below then holds up to that tolerance.

The term $R(g_{\mathcal{G}}) = \inf_{g \in \mathcal{G}} D(\mathbb{P}^* \,\|\, \mathbb{P}_g)$ is the *approximation error*, the distance that even the best generator of the class keeps from the data. The circle experiment shows no sign that it limits the result. With two latent dimensions on 200 points, width 4 reaches about the same test loss as the widest generator.

Next comes the *surrogate error*, $R(g_S) - R(g_{\mathcal{G}})$. Some distances D cannot be computed even with infinite data. The KL divergence that maximum likelihood minimizes is one of them for the model of Chapter 3, whose likelihood is an integral over the latent variable with no closed form. Chapter 3 therefore maximizes a lower bound on the likelihood in its place. In general the objective S replaces D by something computable. Since g_S minimizes this stand-in in place of D, it can be worse than $g_{\mathcal{G}}$. This term is never negative, since $g_{\mathcal{G}}$ minimizes R. In the circle experiment S and D are the same squared MMD, so the term is zero.

The *statistical error* $R(g_{S_n}) - R(g_S)$ comes from minimizing S_n in place of S, because the samples show $\mathbb{P}^*$ only through n points. The runs on 12 and 200 points show it. With the same architecture, the test loss falls from about 0.4 on 12 points to about 0.02 on 200. The test loss measures $R(\widehat{g})$, so this drop mixes the statistical term with the optimization term below. Separating them needs $R(g_{S_n})$, which we cannot measure because training stops before it reaches g_{S_n}.

Last is the *optimization error*, $R(\widehat{g}) - R(g_{S_n})$, which appears when training stops before reaching g_{S_n}.

The optimization term can be negative, which looks odd in a list of errors. It happens because g_{S_n} is chosen by S_n alone, so nothing forces $\widehat{g}$ to be worse than g_{S_n} when both are measured by R.

On 12 points the circle runs show what stopping early can do. In all three draws the wide generator had a lower test loss at $6{,}000$ steps than at $100{,}000$, because S_n keeps pulling it toward the training points. Measured by R, a generator that minimizes S_n less closely can therefore be the better one. Telling the optimization term apart from the statistical term is beyond what the circle runs can show.

Going further

When the statistical term can be negative

The statistical term can be negative too, but it has less freedom. Every generator of the class has $R(g) \geq R(g_{\mathcal{G}})$, in particular g_{S_n}. Subtracting $R(g_S)$ from both sides gives $R(g_{S_n}) - R(g_S) \geq -\big[R(g_S) - R(g_{\mathcal{G}})\big]$. The right side is the surrogate term with a minus sign. When the surrogate term is zero this says the statistical term is at least zero, so it can be negative only when the surrogate term is positive, and by at most that amount. In the circle experiment the surrogate term is zero, so the statistical term is not negative.

Chapter 2 sets optimization aside and bounds the surrogate and statistical terms together by one inequality.

Check yourself

1. The empirical distribution $\mathbb{P}_n$ fits the training set perfectly. Why is it a bad answer?

   Answer

   > Sampling from it only returns training images, so it never produces a new one. The goal is to be close to $\mathbb{P}^*$, of which the training set is only a finite sample.

2. Can a generator $g : \mathbb{R}^2 \to \mathbb{R}^3$ built from linear layers and ReLU activations define a distribution with a density on $\mathbb{R}^3$?

   Answer

   > No. Such a network is Lipschitz, so by Proposition 6 the set $g(\mathbb{R}^2)$ has volume zero in $\mathbb{R}^3$. All the mass sits on that set, which no density allows.

3. MNIST has an intrinsic dimension between 7 and 13. What does that suggest about the size of the latent space?

   Answer

   > A latent space of about 7 to 13 dimensions can be enough, far below the 784 pixels, since the generator only has to move along the manifold. This does not say how many dimensions training needs.

4. In the circle experiment, which source of error limits the width-1024 generator trained on 12 points, and what fixed it?

   Answer

   > Statistical error. Its class can follow the circle, as the runs on 200 points show, but 12 points do not show that the data lie on a circle. Its training loss is even lower than that of the true circle on the same points. More data fixed it, while longer training made it worse.

5. On 12 points, the width-1024 generator had a better test loss after $6{,}000$ steps than after $100{,}000$. Why?

   Answer

   > The objective is minimized near the copying model, so training longer moves the generator toward the 12 points and away from the circle. Stopping early leaves some optimization error, which here keeps it closer to $\mathbb{P}^*$.

Summary

We started with a model that fits its training set perfectly and teaches us nothing, the copying model. A generator should give us the distribution $\mathbb{P}^*$ behind the samples, with nothing to go on except the samples themselves.

To represent $\mathbb{P}^*$, a generator pushes noise through a network. One latent number was enough to produce the circle exactly. Natural images also lie near a low-dimensional manifold, so a generator can work with about as many latent dimensions as the manifold has, far fewer than the number of pixels. This economy creates a problem. A network with fewer latent dimensions than pixels puts all its probability on a set of zero volume, so it has no density and cannot be trained by maximum likelihood as it stands.

Training brought a second problem. On 12 points every generator fitted the points better than the true circle did, the wider one copied more of them, and longer training pushed it further toward copying. With 200 points the same generators followed the circle. Failures like these trace back to four sources. They are the model class, the objective, the finite sample and the optimization.

All of this leaves the number that training minimizes still to be chosen. It has to say how far $\mathbb{P}_g$ is from $\mathbb{P}^*$, it has to be computed from n samples, and it has to make sense when $\mathbb{P}_g$ has no density. Chapter 2 compares the standard choices and shows which of them still work.

Arjovsky, Martin, and Léon Bottou. 2017. “Towards Principled Methods for Training Generative Adversarial Networks.” In *International Conference on Learning Representations (ICLR)*.

Carlini, Nicholas, Jamie Hayes, Milad Nasr, Matthew Jagielski, Vikash Sehwag, Florian Tramèr, Borja Balle, Daphne Ippolito, and Eric Wallace. 2023. “Extracting Training Data from Diffusion Models.” In *USENIX Security Symposium*.

Dinh, Laurent, Jascha Sohl-Dickstein, and Samy Bengio. 2017. “Density Estimation Using Real NVP.” In *International Conference on Learning Representations (ICLR)*.

Hyvärinen, Aapo. 2005. “Estimation of Non-Normalized Statistical Models by Score Matching.” *Journal of Machine Learning Research* 6: 695–709.

Meehan, Casey, Kamalika Chaudhuri, and Sanjoy Dasgupta. 2020. “A Non-Parametric Test to Detect Data-Copying in Generative Models.” In *International Conference on Artificial Intelligence and Statistics (AISTATS)*.

Oord, Aäron van den, Nal Kalchbrenner, and Koray Kavukcuoglu. 2016. “Pixel Recurrent Neural Networks.” In *International Conference on Machine Learning (ICML)*.

Pope, Phillip, Chen Zhu, Ahmed Abdelkader, Micah Goldblum, and Tom Goldstein. 2021. “The Intrinsic Dimension of Images and Its Impact on Learning.” In *International Conference on Learning Representations (ICLR)*.

Rezende, Danilo Jimenez, and Shakir Mohamed. 2015. “Variational Inference with Normalizing Flows.” In *International Conference on Machine Learning (ICML)*.

Somepalli, Gowthami, Vasu Singla, Micah Goldblum, Jonas Geiping, and Tom Goldstein. 2023. “Diffusion Art or Digital Forgery? Investigating Data Replication in Diffusion Models.” In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Song, Yang, and Stefano Ermon. 2019. “Generative Modeling by Estimating Gradients of the Data Distribution.” In *Advances in Neural Information Processing Systems (NeurIPS)*.

Theis, Lucas, Aäron van den Oord, and Matthias Bethge. 2016. “A Note on the Evaluation of Generative Models.” In *International Conference on Learning Representations (ICLR)*.

Vincent, Pascal. 2011. “A Connection Between Score Matching and Denoising Autoencoders.” *Neural Computation* 23 (7): 1661–74.

---

1. Table 1 of Pope et al. (2021). The ImageNet resolution is stated in their introduction. Each range comes from the number of neighbours used by their estimator.

❝ Cite this chapter

Bouftini, I. (2026). Learning a distribution. Chapter 1 of the course Generative Models for Images. https://ibouftini.github.io/courses/generative-models-for-images/learning-a-distribution/

```
@misc{bouftini2026learning,
  author = {Bouftini, Imade},
  title  = {Learning a distribution},
  year   = {2026},
  note   = {Chapter 1 of the course Generative Models for Images},
  url    = {https://ibouftini.github.io/courses/generative-models-for-images/learning-
a-distribution/}
}
```

Printed from https://ibouftini.github.io/courses/generative-models-for-images/learning-a-distribution/. The online version has the interactive figures and any later corrections.

© 2026 Imade Bouftini GitHub LinkedIn